

**BABCOCK UNIVERSITY
SCHOOL OF COMPUTING AND ENGINEERING SCIENCES
COMPUTER SCIENCE**

2025/2026 ACADEMIC SESSION

**A RESEARCH PROJECT PROPOSAL PAPER ON

DESIGN AND IMPLEMENTATION OF A LOG ANALYSIS AND
EVENT-DRIVEN AUTO-RECOVERY SYSTEM FOR A FINANCIAL
MICROSERVICE INFRASTRUCTURE**

BY

**ABU SOLOMON E.
PG/25/0548**

31st MARCH, 2026

CHAPTER ONE

INTRODUCTION

1.1 Background of Study

Unplanned system downtime costs organizations an average of \$5,600 per minute, with 47% of financial institutions losing over \$1 million per hour of disruption [1]. As financial systems grow more complex, resilience becomes an operational imperative rather than an engineering preference. Modern financial systems have widely embraced microservices architectures to achieve the scalability and agility demanded by contemporary markets [2], yet this shift has introduced significant operational complexity. Distributed systems generate enormous volumes of telemetry data that offer only fragmented visibility into systemic behaviour [4], a challenge illustrated by Islam et al. [5], whose monitoring dataset spanned 39,365 rows and 117,448 columns, demonstrating how modern telemetry can overwhelm both human operators and traditional analytical methods.

The theoretical foundation for self-healing systems traces to Minsky [6], who established that invariant properties must be deliberately imposed through architectural constraints to sustain system integrity across all states, including failure. This insight underpins IBM's Autonomic Computing vision and the MAPE-K (Monitor, Analyze, Plan, Execute, Knowledge) feedback loop for automated recovery [7, 8]. Event-Driven Architecture has since emerged as a powerful enabler of these principles, with Beereddy [1] identifying five core resilience patterns namely, circuit breakers, buffer-based event preservation, automatic failure detection, event replay, and reconciliation protocols, that achieved Mean Time to Recovery (MTTR) reductions of 6.8× over manual recovery, rising to 12.3× for complex failures. Event-driven communication is particularly suited to finance, where transactions naturally flow as cascading event sequences and event sourcing simultaneously satisfies audit trail requirements [3].

This research proposes combining event-driven recovery orchestration with rule-based log analysis, delivering a practical, integrated, and auditable solution leveraging Prometheus, Grafana Loki, and Spring Boot to operationalize the MAPE-K loop. Regulated financial

environments favor rule-based methods that are industrially preferred over machine learning for their interpretability and lower false positive rates [9, 10].

1.2 Statement Of Problem

Financial microservices environments demand 99.99% uptime and comprehensive audit trails for regulatory compliance, yet remain critically dependent on manual fault remediation. The average Mean Time to Recovery (MTTR) for streaming applications stands at 76 minutes, with 91% of recovery operations requiring human involvement [1], inflating costs, causing alert fatigue, and delaying remediation as detected anomalies await human response.

While automated approaches have been explored, machine learning-based methods require extensive labelled training data, exhibit non-deterministic behaviour, and conflict with regulators' demands for explainable and auditable system behaviour [7, 10], rendering them impractical in regulated financial contexts.

This gap necessitates a practical, explainable, and automatically auditable system that unifies real-time fault detection with autonomous remediation under a structured feedback loop, providing financial institutions with a deployable reference architecture that satisfies both availability and regulatory transparency requirements.

1.3 Aim and Objectives

This project is aimed at designing and implementing an intelligent auto-recovery system for financial microservices that leverages log analysis and event-driven architecture to automatically detect anomalies and execute remediation actions, to reduce Mean Time to Recovery (MTTR) and improve overall system resilience. The objectives of the project are as follows:

1. To design a comprehensive auto-recovery architecture that facilitates fault identification and recovery for financial microservices environments.
2. To implement the designed system using cloud-native tools with an event-driven recovery engine capable of executing automated remediation actions.

3. To test and evaluate the system's effectiveness by measuring recovery time, fault detection accuracy, false positive rates, and improvement in system efficiency relative to manual intervention baselines.

1.4 Justification of Study

There has been extensive research on monitoring, observability, and resilience patterns in microservices, significant gaps remain in the literature that this study seeks to address. Existing work tends to treat failure detection and recovery as separate concerns, focusing either on monitoring and observability or on remediation mechanisms in isolation. There is a notable absence of research on integrated systems that cohesively combine log analysis, metrics monitoring, and automated remediation, particularly within the context of financial microservices [13]. Beyond this, most available research remains largely theoretical, offering architectural patterns and conceptual models without providing comprehensive implementation frameworks using modern cloud-native tools. Practitioners seeking concrete guidance on integrating technologies such as Spring Boot, Prometheus, and Grafana Loki for automated recovery find little actionable direction in current literature.

A further gap exists in both domain specificity and recovery design. General-purpose auto-recovery systems do not account for the unique constraints of financial environments, including regulatory compliance, audit trail maintenance, transaction consistency, and the demand for explainable recovery actions. The prevailing reliance on machine learning for anomaly detection further compounds this, as ML-based approaches introduce complexity, require extensive training data, and lack the transparency that financial regulators require leaving the potential of interpretable, rule-based approaches largely unexplored. Critically, the application of event-driven architectures to infrastructure-level fault recovery remains underresearched; coordinating detection, decision-making, action execution, and validation through structured event flows presents a promising but insufficiently studied approach to building resilient financial microservices systems.

1.5 Significance of Study

The significance of this study lies in its direct response to the need for an explainable, auditable and practical implementation structure that integrates autonomous remediation and addresses critical operational and financial challenges presented by modern financial microservices architectures. Unplanned system downtime in financial institutions can cost over \$1 million per hour, and manual fault remediation currently leads to an average Mean Time to Recover (MTTR) of 76 minutes, with 91% of recovery operations requiring human intervention. This research proposes an intelligent auto-recovery system designed to dramatically reduce MTTR by 80–90% through automated detection and remediation, directly mitigating severe financial losses and helping institutions meet stringent 99.99% uptime Service Level Agreements (SLAs). By operationalizing the MAPE-K (Monitor, Analyze, Plan, Execute, Knowledge) feedback loop, the system transforms a reactive, cost-inflating process into a proactive, resilient, and efficient operation.

Furthermore, the study makes a significant architectural and regulatory contribution by employing a rule-based log analysis approach. This method is deliberately chosen over machine learning to ensure the system's actions are interpretable, auditable, and maintain low false-positive rates, qualities that are essential for regulatory compliance in the financial sector. Manual recovery processes often result in data loss or duplication in 78% of streaming applications, which risks incorrect balances and legal liability. By developing an event-driven recovery engine using a transparent observability stack (Prometheus, Grafana Loki, and Spring Boot), this research provides a practical implementation framework for organizations to build auditable, self-healing systems that maintain the processing integrity and transparency required for high-volume, regulated financial operations.

1.6 Scope and Limitation

This study focuses on the design, development, and evaluation of a self-healing microservices architecture within a simulated financial services environment. The implementation integrates automated failure detection, diagnosis, and recovery mechanisms across a controlled cluster of microservices, tested against common failure patterns including crashes, high error rates,

resource exhaustion, and connectivity issues. The scope is bound to this simulated context, using a defined stack of Spring Boot, Prometheus, and Loki, with performance assessed through standard reliability and recovery metrics.

Several limitations apply. The use of a simulated environment means the system may not fully reflect the complexities of production financial systems, including legacy dependencies, regulatory integration, and enterprise-scale loads. The study addresses a defined set of failure scenarios and does not cover all failure modes, such as security breaches, data corruption, network partitions, or split-brain scenarios. The implementation is also technology-specific, and findings may not translate directly to other stacks or monitoring frameworks. Additionally, the evaluation covers a limited timeframe, leaving long-term stability and rare failure scenarios unassessed. While audit trail capabilities are incorporated, full regulatory compliance is not validated, as this requires live deployment and engagement with regulatory bodies. These constraints are acknowledged and do not undermine the validity of findings within the defined scope.

1.7 Methodology

The first objective which is to design a comprehensive auto-recovery architecture for financial microservices environments is addressed in **Phase 1**. This phase establishes the theoretical and architectural foundation of the study through a comprehensive review of academic papers, technical documentation, and industry reports covering self-healing systems, microservices resilience, observability, log analysis, and event-driven architecture. Alongside this, existing monitoring and auto-recovery solutions, including commercial platforms such as Datadog and New Relic as well as open-source tools including the ELK Stack, Prometheus Operator, and Spring Boot Admin, are analysed to identify design opportunities and inform architectural decisions. From this foundation, a high-level system architecture is produced, encompassing component interaction diagrams, data flow models for log and metrics pipelines, event-driven recovery workflow sequences, and integration patterns between Spring Boot services, Prometheus, and Loki. A recovery decision matrix is also defined, mapping detected failure conditions including service crashes, high error rates, response time degradation, database

connection pool exhaustion, memory leaks, and cascading failures, to appropriate remediation strategies such as service restarts, circuit breaker activation, connection pool resets, and traffic rerouting.

The second objective is to implement the designed system using cloud-native tools with an event-driven recovery engine capable of executing automated remediation actions. This is fulfilled across Phases 2 and 3. Phase 2 establishes the development and monitoring infrastructure by configuring Prometheus for metrics collection, including scrape configurations, custom recording rules, and Alertmanager routing, and deploying Grafana Loki alongside Promtail agents for log aggregation, label extraction, and LogQL-based failure pattern detection. Grafana dashboards are configured to provide visibility into service health, error rates, and latency percentiles, with all components orchestrated via Docker Compose. Phase 3 builds directly upon this infrastructure by developing three to four representative financial microservices which include a Payment Processing Service, Account and Transaction Management Service, Notification Worker Service, and API Gateway, using Spring Boot 3.x. Each service incorporates Spring Boot Actuator for health exposure, Micrometer for Prometheus metrics, structured JSON logging with correlation IDs for distributed tracing, Resilience4j for circuit breaker implementation, and configurable failure injection for controlled testing. The auto-recovery engine is subsequently implemented as three integrated components: a Detection Module that aggregates Prometheus and Loki alert rules across metrics-based and log pattern-based anomalies; an Event Bus responsible for routing recovery events using Spring Application Events or a lightweight message broker such as RabbitMQ or Kafka; and a Recovery Action Executor that programmatically triggers remediation actions including service restarts via Spring Boot Admin, circuit breaker control through the Resilience4j API, database connection pool resets, traffic rerouting through API Gateway reconfiguration, and fallback mechanisms such as cached responses. A feedback loop incorporating post-recovery health checks, audit logging, and loop-prevention controls completes the engine.

The third objective, which is to test and evaluate the system's effectiveness by measuring recovery time, fault detection accuracy, false positive rates, and improvement in system

efficiency relative to manual intervention baselines, is addressed in Phase 4. Functional testing first verifies detection accuracy, alert triggering, recovery action execution, post-recovery validation, and audit trail completeness across each defined failure scenario. Performance testing then measures Mean Time to Recovery (MTTR) for six controlled failure scenarios namely, service crash, high error rate, response time degradation, database connection pool exhaustion, memory leak, and cascading multi-service failure, with each repeated ten times to ensure statistical validity, alongside detection latency, execution latency, and monitoring system overhead. A comparative analysis benchmarks automated recovery times against simulated manual intervention baselines, and evaluates false positive and false negative detection rates as well as the completeness of service restoration. Quantitative data including recovery times, detection accuracy, and resource utilisation are subjected to appropriate statistical analysis, including t-tests for MTTR comparison, with findings presented through graphs, tables, and narrative interpretation to substantiate the research conclusions.

CHAPTER TWO

LITERATURE REVIEW

... In progress

CHAPTER THREE

METHODOLOGY

3.1 Introduction

This chapter presents a comprehensive account of the methodology employed in the design and implementation of a log analysis and event-driven auto-recovery system for a financial microservices infrastructure. The chapter describes the research design, the system architecture governing the interaction of its components, the financial microservices tier developed to simulate a realistic operational environment, the observability and telemetry infrastructure responsible for continuous data collection and anomaly detection, the auto-recovery engine that translates detected faults into targeted remediation actions, the experimental evaluation methodology used to validate system performance, and the development tools and technologies used throughout the implementation. The chapter also documents the recovery decision logic, audit trail mechanisms, and post-recovery validation procedures incorporated to satisfy the explainability and regulatory transparency requirements that characterise financial services deployments.

3.2 Research Design

This research involves the systematic design, construction, and experimental evaluation of an intelligent auto-recovery system for financial microservices, developed to demonstrate that rule-based log analysis and event-driven recovery orchestration can significantly reduce Mean Time to Recovery (MTTR) while maintaining the auditability required in regulated financial environments. The research is quantitative and experimental in nature, with performance measured through controlled fault injection experiments against pre-defined metrics: Mean Time to Recovery, fault detection accuracy, false positive rate, and monitoring system overhead.

The work is organised across four sequential phases, each producing concrete deliverables that build directly on the preceding phase. Phase 1 establishes the architectural foundation through a review of relevant literature and existing tools, culminating in a high-level system architecture and a recovery decision matrix. Phase 2 deploys and configures the observability infrastructure, including metrics collection, log aggregation, and alerting pipelines. Phase 3 implements the

financial microservices tier and the auto-recovery engine atop that infrastructure. Phase 4 subjects the completed system to functional and performance testing against six defined failure scenarios, with results benchmarked against simulated manual intervention baselines. Table 3.1 maps each phase to its specific activities and deliverables.

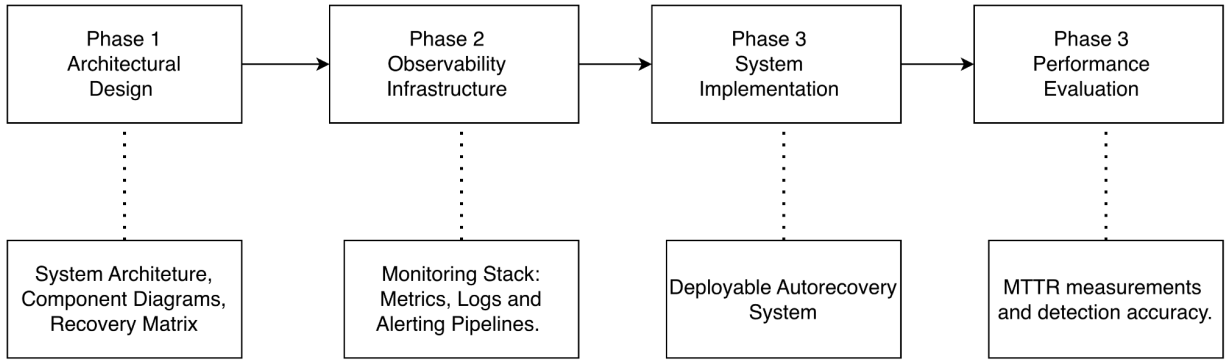


Figure 3.1: Research phases as applied in this study

3.3 Phase 1: Architectural Foundation

3.3.1 Literature Review and Tool Analysis

A focused review of the academic and technical literature was conducted to ground the system design in established practice. The review covered self-healing systems and autonomic computing [15], microservices resilience patterns [2], distributed systems observability [16], and event-driven architecture for fault recovery [17]. From this review, the MAPE-K feedback loop, originally formalised by Kephart and Chess [15], was identified as the most appropriate architectural model for the auto-recovery engine, providing a structured decomposition into Monitor, Analyse, Plan, and Execute components.

The tool selection was additionally informed by a comparative analysis of existing observability platforms. Commercial solutions, including Datadog and New Relic, were evaluated but excluded on the grounds that they introduce licensing constraints incompatible with a fully reproducible open-source research environment. The ELK Stack (Elasticsearch, Logstash, Kibana) was similarly considered; while capable, its resource demands and operational complexity are disproportionate for a local simulation environment of the scale used in this study. The selected open-source stack, summarised in Table 3.2, was chosen because each tool

integrates natively with Spring Boot, incurs minimal resource overhead in a local Docker Compose deployment, and collectively satisfies all observability, alerting, and service management requirements of the study.

Table 3.2: Selected open-source toolchain with justification

Tool	Category	Justification
Prometheus	Metrics	Open-source, pull-based metrics system with native Spring Boot Actuator integration via Micrometer. Industry-standard for microservices observability [6].
Grafana Loki	Log aggregation	Lightweight, label-based log aggregation engine designed to complement Prometheus. Avoids the index overhead of ELK while providing LogQL-based pattern detection suitable for alerting [20].
Promtail	Log shipping	Official Loki log-shipping agent. Provides pipeline-based label extraction and structured log forwarding with minimal resource overhead [20].
Grafana	Visualisation	Unified dashboard platform natively supporting both Prometheus and Loki datasources. Enables alert rule definition and recovery activity timelines within a single interface [20].
Spring Boot 3.x	Microservices	Mature, open-source Java framework providing production-ready defaults: Actuator health endpoints, Micrometer metrics, and Resilience4j integration. Widely adopted in financial services applications [2].
Resilience4j	Fault tolerance	Lightweight, functional fault-tolerance library designed specifically for Java 8+ and Spring Boot. Provides circuit breakers, rate limiters, and bulkheads with direct Micrometer instrumentation [21].
RabbitMQ	Messaging	Open-source AMQP-compliant message broker. Selected for its lightweight footprint in local simulation environments and native Spring AMQP support for asynchronous recovery event routing [22].
Spring Boot Admin	Service management	Open-source administrative interface for Spring Boot applications. Exposes management endpoints used by the Recovery Action Executor to programmatically trigger service restarts [23].
Springdoc OpenAPI	API documentation	Automated OpenAPI 3.0 specification generation from Spring MVC annotations. Provides a Swagger UI interface

		that accelerates service integration verification and endpoint documentation [24].
Docker Compose	Orchestration	Declarative multi-container orchestration sufficient for local simulation. Enables reproducible deployment of the full observability and microservices stack without requiring a Kubernetes cluster [25].
PostgreSQL	Database	Open-source relational database. Selected for its robustness in financial transaction workloads and HikariCP connection pool integration with Spring Boot, enabling connection pool exhaustion simulation [2].

Prometheus was selected as the metrics collection backend on account of its pull-based architecture, which is well-suited to containerised environments, and its native integration with Spring Boot Actuator via the Micrometer instrumentation facade [19]. Grafana Loki was chosen for log aggregation in preference to the ELK Stack because it indexes only metadata labels rather than full log content, substantially reducing storage and memory requirements while retaining the LogQL query interface necessary for pattern-based alert rules [20]. Promtail serves as the log-shipping agent, providing pipeline-based label extraction from structured JSON log output. Grafana consolidates both data sources into a single visualisation and alerting interface [1]. The Spring Boot 3.x framework underpins the microservices tier, providing Actuator health endpoints, Micrometer metrics exposition, and first-class Resilience4j circuit breaker integration [2]. Springdoc OpenAPI generates Swagger UI documentation automatically from controller annotations, providing a live API reference that accelerates development and integration verification [24].

3.3.2 System Architecture Design

The system architecture is organised into four functional tiers: the Financial Microservices Tier, the Observability and Telemetry Tier, the Auto-Recovery Engine, and the Audit and Reporting Layer. The Financial Microservices Tier comprises the API Gateway, Account Service, Payment Service, and Notification Worker, each instrumented with Spring Boot Actuator, Micrometer, and Resilience4j. The Observability Tier collects metrics via Prometheus scrape jobs and aggregates structured logs via Promtail and Loki, with Grafana providing unified visualisation. The Auto-Recovery Engine receives alert notifications from both Prometheus Alertmanager and Loki

alert rules via webhook, classifies incoming events, and dispatches remediation actions through the Recovery Action Executor. The Audit Layer records every detection event, recovery action, and post-recovery health check outcome with timestamps and correlation identifiers to support regulatory traceability [18].

Component interaction follows an event-driven pattern. Prometheus and Loki alert webhooks deliver structured payloads to the Detection Module, which deduplicates and classifies alerts before publishing typed recovery events onto the internal Event Bus. Individual Recovery Handlers subscribe to specific event types and delegate to the Recovery Action Executor, which applies the remediation strategy defined in the externalised recovery decision matrix. Post-recovery health checks confirm service restoration before the loop-prevention registry releases the cooldown lock on the affected service.

3.3.3 Recovery Decision Matrix

A recovery decision matrix was defined to provide a deterministic, auditable mapping between detected failure conditions and their corresponding remediation strategies. The matrix covers all six failure scenarios evaluated in Phase 4 and is externalised as a configuration file (recovery-matrix.yml) to allow threshold and action adjustments without code changes. Table 3.3 presents the matrix as implemented.

Table 3.3: Recovery decision matrix

Failure Scenario	Detection Signal	Remediation Action
Service Crash	Spring Boot Actuator health DOWN; HTTP 5xx rate spike	Service restart via Spring Boot Admin; post-restart health check
High Error Rate	HTTP 5xx rate > 10% over 2-minute window (Prometheus)	Circuit breaker OPEN; traffic rerouting to fallback endpoint
Response Time Degradation	p95 latency > 2 s over 3-minute window (Prometheus)	Circuit breaker HALF-OPEN probe; connection pool inspection; cached response fallback
DB Connection Pool Exhaustion	HikariCP pending threads > threshold; connection timeout errors in logs (Loki)	Connection pool reset via Actuator endpoint; query rate limiting

Memory Leak	JVM heap usage > 85% sustained for 5 minutes (Prometheus JVM metrics)	Graceful service restart; heap dump capture for post-mortem audit
Cascading Multi-Service Failure	Correlated health degradation across two or more services within 60-second window	Coordinated circuit breaker activation; traffic isolation; sequential recovery with health gating

The primary remediation action is applied first; if the post-recovery health check fails within the configured timeout, the secondary action is triggered. Cascading failure scenarios additionally enforce sequential recovery ordering, whereby each service is confirmed healthy before recovery of the next dependent service proceeds.

3.4 Phase 2: Observability Infrastructure

3.4.1 Infrastructure Overview and Local Deployment Strategy

The entire observability stack is deployed locally using Docker Compose, providing a fully reproducible simulation environment that requires no external cloud dependencies. All infrastructure configuration is externalised into a dedicated config/ directory tree, with each tool's configuration files mounted as read-only volumes into the corresponding container. This approach separates configuration concerns from container image definitions and permits configuration changes without image rebuilds.

The Docker Compose topology defines network-isolated services for Prometheus, Grafana Loki, Promtail, Grafana, Spring Boot Admin, RabbitMQ, and PostgreSQL, alongside the four application microservices and the Recovery Engine. A shared monitoring network allows all microservices to be scraped by Prometheus and to forward logs to Loki, while a separate application network isolates inter-service HTTP communication.

3.4.2 Metrics Collection with Prometheus

Prometheus is configured with a scrape interval of fifteen seconds for all microservice targets. Each Spring Boot application exposes the /actuator/prometheus endpoint, from which Prometheus pulls Micrometer-instrumented metrics including JVM heap utilisation, HTTP request rates and latency histograms, HikariCP connection pool statistics, and Resilience4j

circuit breaker state metrics [6]. Custom recording rules pre-compute derived metrics, including per-service HTTP error rate ratios and p95 latency aggregations, to reduce query complexity at alert evaluation time.

Alertmanager is configured to receive Prometheus alert rule firings and route them to the Recovery Engine's webhook receiver endpoint. Alert grouping is configured by service name and alert type to prevent notification flooding during multi-service incidents. A repeat interval of five minutes and a group wait of thirty seconds balance alert responsiveness against deduplication [19].

3.4.3 Log Aggregation with Grafana Loki and Promtail

All microservices emit structured JSON logs via Logback, with each log entry including at minimum a timestamp, log level, service name, trace identifier, and correlation ID injected by the per-request CorrelationIdFilter. Promtail is deployed as a sidecar-pattern container in the Docker Compose topology, with one Promtail instance configured per microservice. Each Promtail configuration defines a pipeline stage that parses the JSON log stream, extracts the service, level, and traceId fields as Loki labels, and forwards labelled log streams to the Loki ingestion endpoint [20].

Loki alert rules are defined using LogQL metric queries that detect sustained error log rates and specific exception patterns, including database connection timeout exceptions and out-of-memory errors. These alert rules are evaluated at a thirty-second interval and fire webhooks to the Recovery Engine's Loki alert receiver endpoint upon threshold breach.

3.4.4 Grafana Dashboard and Alert Configuration

Three Grafana dashboards are provisioned automatically from JSON definition files at container startup. The System Health Overview dashboard presents per-service UP/DOWN status, HTTP request rate, error rate percentage, and p95 latency. The Infrastructure Metrics dashboard displays JVM heap usage, garbage collection pause duration, HikariCP pool utilisation, and thread count per service. The Recovery Activity dashboard presents a timeline of recovery events drawn from the audit log, including detection-to-action latency and recovery outcome per incident [1].

Grafana contact points are configured to deliver alert notifications to a Slack-compatible webhook endpoint for operator visibility during experiments, supplementing the programmatic webhook receiver in the Recovery Engine.

3.5 Phase 3: System Implementation

3.5.1 Financial Microservices Tier

3.5.1.1 Service Overview and Project Structure

The financial microservices tier is implemented as a multi-module Maven project. A shared common-core module defines common entity models (UserEntity, AccountEntity, TransactionEntity, PaymentEntity), shared DTOs, a unified exception handling framework via GlobalExceptionHandler, and centralized OpenAPI configuration. This structure avoids duplication of domain model definitions across services and ensures consistent API response schemas. The four application services, namely, api-gateway, account-service, payment-service, and notification-worker, all depend on common-core and are packaged as independent deployable JARs, each with its own Dockerfile.

3.5.1.2 Common Service Instrumentation

A consistent instrumentation baseline is applied across all services. Spring Boot Actuator exposes the /actuator/health, /actuator/prometheus, /actuator/info, and /actuator/metrics endpoints. Micrometer is configured with a Prometheus registry, providing automatic instrumentation of HTTP server requests, JVM memory pools, garbage collection, and thread utilisation [19]. Each service configures a CorrelationIdFilter that extracts or generates a UUID-based correlation ID from incoming requests, propagates it via MDC, and includes it in all downstream HTTP calls via a WebClient exchange filter. This ensures that a single business transaction can be traced across service boundaries in both metrics and log data.

Resilience4j circuit breakers are configured on all inter-service HTTP calls, with state transition events exposed as Micrometer metrics. The circuit breaker configuration specifies a sliding window of ten calls, a failure rate threshold of fifty percent for OPEN transition, a wait duration in OPEN state of thirty seconds, and two permitted calls in HALF-OPEN state [21]. These thresholds align with the detection signals defined in the recovery decision matrix.

3.5.1.3 API Gateway

The API Gateway is implemented using Spring Cloud Gateway. It serves as the single ingress point for all external requests, performing JWT validation via a `JwtAuthFilter` before forwarding requests to downstream services. Route definitions in `RoutingConfig` map URI prefixes to the `account-service` and `payment-service`. A `ResilienceConfig` applies a `Resilience4j` circuit breaker to each route, with fallback responses served by `FallbackController`. A `HealthIndicatorConfig` aggregates downstream service health into a composite health indicator, enabling the gateway's `/actuator/health` endpoint to reflect the health of the services it fronts.

3.5.1.4 Account Service

The Account Service provides user registration, authentication, account management, and transaction query functionality. `AuthController` exposes `/auth/register` and `/auth/login` endpoints, delegating to `AccountService` for credential validation and JWT issuance. `AccountController` exposes CRUD operations over account resources, and `TransactionController` exposes endpoints for transaction history queries. An `OTPService` provides one-time password generation and validation for sensitive operations. The service persists data to a PostgreSQL database via Spring Data JPA repositories and applies schema migrations via Liquibase. A `FaultSimulationController` exposes endpoints for controlled failure injection during Phase 4 experiments.

3.5.1.5 Payment Service

The Payment Service orchestrates payment transactions. `PaymentController` accepts payment initiation requests, validates the request against the account balance by calling the Account Service via a resilience-wrapped `WebClient`, persists a `PaymentEntity` with a `PENDING` status, and publishes a payment notification event to RabbitMQ via a configured exchange and routing key. Upon successful processing, the payment status is updated to `COMPLETED`; upon failure, it is set to `FAILED` with the exception detail recorded. A `FaultSimulationController` permits injection of processing delays, forced exceptions, and connection pool saturation for experimental purposes.

3.5.1.6 Notification Worker

The Notification Worker is a listener-only service that consumes payment notification messages from RabbitMQ via `PaymentNotificationListener`, which is registered as a `@RabbitListener` on

the configured queue. NotificationService processes the consumed message and logs the notification outcome as a structured JSON entry. The service is deliberately stateless and is designed to demonstrate cascading failure recovery scenarios in which notification backpressure indicates upstream payment processing degradation.

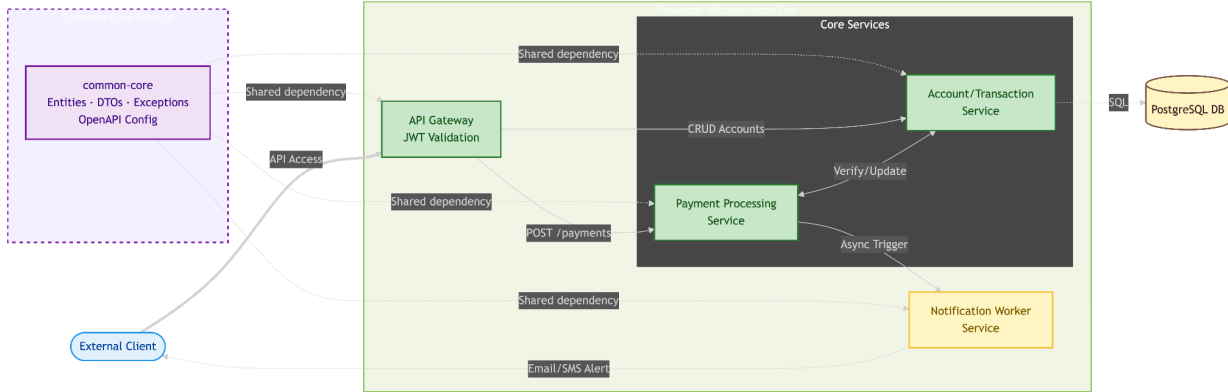


Figure 3.2: Microservice Architecture

3.5.1.7 Failure Injection Mechanism

Each service exposes a FaultSimulationController providing endpoints to activate named fault profiles: artificial response delay injection via Thread.sleep, forced exception throwing, memory pressure simulation via large object allocation, and connection pool saturation via concurrent blocking database queries. Fault profiles are activated and deactivated via HTTP POST requests, enabling precise experimental control over fault onset timing without requiring service restarts. This approach is consistent with the principles of controlled chaos engineering as described by Basiri et al. [22].

3.5.2 Auto-Recovery Engine

3.5.2.1 Architecture and MAPE-K Mapping

The Recovery Engine is implemented as a standalone Spring Boot application. Its internal architecture follows the MAPE-K autonomic computing reference model [15]: the Detection Module fulfils the Monitor and Analyse functions; the Event Bus fulfils the Knowledge distribution function; and the Recovery Action Executor fulfils the Plan and Execute functions.

This decomposition produces clearly bounded components with well-defined interfaces, supporting independent testability and future extension.

3.5.2.2 Detection Module

The Detection Module comprises two webhook controllers and a classification layer. AlertmanagerWebhookController receives Prometheus Alertmanager webhook payloads and AlertmanagerWebhookController receives Loki alert rule webhook payloads. Both controllers delegate to AlertClassifier, which maps alert names to typed recovery event classes (ServiceCrashEvent, HighErrorRateEvent, LatencyDegradationEvent, DbPoolExhaustionEvent, MemoryLeakEvent, CascadingFailureEvent) using the mappings defined in the externalised recovery-matrix.yml. An EventDeduplicationCache, implemented as a time-bounded in-memory cache, suppresses duplicate alert firings for the same service and alert type within a configurable deduplication window, preventing redundant recovery actions during alert flapping.

3.5.2.3 Event Bus

The Event Bus is implemented as a thin wrapper around Spring's ApplicationEventPublisher, providing typed publish and subscribe semantics via the EventBus component. Recovery events are published synchronously within the Spring application context, with each RecoveryHandler registered as an @EventListener on its corresponding event type. This approach avoids the operational overhead of an external broker within the engine itself while retaining the decoupling benefits of event-driven design. The EventBus abstraction is designed to permit substitution with a RabbitMQ publisher in future work without changes to handler implementations.

3.5.2.4 Recovery Action Executor

RecoveryActionExecutor orchestrates the execution of remediation actions by consulting the recovery decision matrix for the primary and secondary actions applicable to the received event type and affected service. Actions are implemented as discrete, injectable components: ServiceRestartAction invokes the Spring Boot Admin REST API to restart the target application instance; CircuitBreakerControlAction calls the Resilience4j Actuator endpoint on the target service to force a circuit breaker state transition; ConnectionPoolResetAction invokes a custom Actuator endpoint on the target service to reset the HikariCP pool; and TrafficRerouteAction modifies the API Gateway route configuration to redirect traffic away from the degraded service.

Following primary action execution, HealthCheckPoller polls the target service's /actuator/health endpoint at five-second intervals for up to sixty seconds to confirm recovery before the audit record is closed.

3.5.2.5 Audit Logging and Loop Prevention

AuditLogger records a structured AuditEvent for every recovery lifecycle transition: alert received, event classified, action dispatched, action completed, health check passed or failed. Each audit event carries the correlation ID of the originating alert, the affected service identifier, the action type, and a UTC timestamp. Audit events are persisted to a dedicated audit log file as newline-delimited JSON, ensuring that the full recovery timeline is available for post-experiment analysis and satisfies the audit trail requirements of financial services environments [18].

LoopPreventionRegistry maintains a cooldown map keyed by service identifier. Once a recovery action is dispatched for a service, that service is locked for a configurable cooldown period (default: 120 seconds), during which subsequent alerts for the same service are acknowledged but not actioned. This prevents recovery action storms in scenarios where the root cause persists beyond the first remediation attempt and ensures that repeated alerting does not exhaust service restart quotas.

3.6 Phase 4: Experimental Evaluation

3.6.1 Evaluation Strategy

The evaluation is conducted in two sequential stages. Functional testing first verifies the correctness of the end-to-end detection and recovery pipeline for each defined failure scenario. Performance testing then quantifies system behaviour under each scenario across repeated experimental trials. Both stages are conducted in the local Docker Compose environment, with all external network traffic disabled to ensure result reproducibility.

3.6.2 Failure Scenario Definitions

Six controlled failure scenarios are evaluated, each corresponding to a row in the recovery decision matrix. Service crash is induced by sending a SIGTERM signal to the target container process via the fault injection endpoint. High error rate is induced by activating the forced-exception fault profile, which causes all incoming requests to return HTTP 500 responses.

Response time degradation is induced by activating the delay injection profile with a configured sleep duration exceeding the p95 latency alert threshold. Database connection pool exhaustion is induced by activating the pool saturation profile, which opens and holds a configurable number of connections. Memory leak simulation is induced by activating the heap pressure profile, which allocates and retains byte arrays until the JVM heap utilisation threshold is breached. Cascading multi-service failure is induced by simultaneously activating the error rate profile on both the Payment Service and the Account Service, creating correlated alert conditions.

3.6.3 Functional Testing

For each scenario, functional testing verifies that: (1) the relevant Prometheus or Loki alert rule fires within the expected detection window; (2) the Alertmanager or Loki webhook payload is correctly received and classified by the Detection Module; (3) the appropriate recovery action is dispatched by the Recovery Action Executor; (4) the post-recovery health check confirms service restoration; and (5) a complete, timestamped audit trail is recorded. Pass/fail outcomes are recorded for each verification criterion per scenario.

3.6.4 Performance Testing and MTTR Measurement

Each failure scenario is repeated ten times to provide a sample size sufficient for parametric statistical analysis. For each trial, four timing measurements are recorded:

1. T_{inject} : the timestamp at which the fault is activated;
2. T_{detect} : the timestamp at which the Recovery Engine receives the alert webhook;
3. T_{action} : the timestamp at which the primary remediation action completes.
4. $T_{recover}$: the timestamp at which the post-recovery health check returns a healthy status.

MTTR is defined as $T_{recover}$ minus T_{inject} . Detection latency is defined as T_{detect} minus T_{inject} . Execution latency is defined as T_{action} minus T_{detect} . All timestamps are captured from audit log entries to microsecond precision.

Monitoring system overhead is measured as the additional CPU and memory utilisation attributable to the Prometheus and Loki stack relative to a baseline deployment without observability components, sampled at ten-second intervals throughout each experimental trial using Docker container statistics.

3.6.5 Baseline Comparison

A simulated manual intervention baseline is constructed for each failure scenario by measuring the time from fault activation to service restoration under a human-operated recovery procedure. The procedure is standardised as: (1) operator notices the alert in Grafana (simulated as a fixed thirty-second detection delay representing the lower bound of human alert response time [16]); (2) operator identifies the affected service and executes the prescribed remediation command; (3) operator confirms service restoration via the health dashboard. The manual MTTR baseline for each scenario is the sum of the fixed detection delay and the measured execution time for the manual remediation command. Automated MTTR results are compared against these baselines to quantify recovery time improvement.

3.6.6 Statistical Analysis

Descriptive statistics (mean, standard deviation, minimum, maximum) are computed for MTTR, detection latency, and execution latency across the ten trials per scenario. A paired t-test is applied to compare automated and manual MTTR distributions per scenario, with a significance level of 0.05 [24]. Effect size is reported as Cohen's d to complement the p-value with a measure of practical significance. Results are presented in tabular form with supporting boxplots and interpreted in the context of the research objectives.

3.7 Development Tools and Environment

Table 3.4 summarises the complete development toolchain used throughout the implementation. All tools are open-source unless otherwise noted.

Table 3.4: Development tools and environment

Category	Tool / Technology
Microservices framework	Spring Boot 3.x / Java 21
Metrics collection	Prometheus 2.x, Micrometer

Log aggregation	Grafana Loki, Promtail
Visualisation & alerting	Grafana
Circuit breaker	Resilience4j
Messaging / Event Bus	RabbitMQ
Service management	Spring Boot Admin
API documentation	Springdoc OpenAPI (Swagger UI)
Database	PostgreSQL 15, HikariCP
Containerisation	Docker, Docker Compose
Build tool	Apache Maven
Version control	Git / GitHub

3.8 Summary

This chapter has presented the complete methodology for the design, implementation, and evaluation of the auto-recovery system. Phase 1 established the architectural and theoretical foundation, producing the system architecture and recovery decision matrix. Phase 2 deployed the open-source observability infrastructure, providing the metrics collection, log aggregation, and alerting pipelines upon which the recovery engine depends. Phase 3 implemented the financial microservices tier and the three-component auto-recovery engine, with audit logging and loop-prevention controls satisfying the transparency requirements of financial deployments. Phase 4 defined the controlled experimental evaluation methodology, including six failure scenarios, MTTR measurement procedures, and statistical analysis protocols. The results of the Phase 4 evaluation are presented in Chapter 4 of this paper.

REFERENCES

- [1] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, Jan. 2003.
- [2] C. Richardson, *Microservices Patterns*. Shelter Island, NY: Manning, 2018.
- [3] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, *Site Reliability Engineering*. Sebastopol, CA: O'Reilly, 2016.
- [4] G. Hohpe and B. Woolf, *Enterprise Integration Patterns*. Boston, MA: Addison-Wesley, 2003.
- [5] M. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed. Raleigh, NC: Pragmatic Bookshelf, 2018.
- [6] Prometheus Authors, "Prometheus: From metrics to insight." [Online]. Available: <https://prometheus.io>. [Accessed: 2024].
- [7] Grafana Labs, "Grafana Loki: Log aggregation system." [Online]. Available: <https://grafana.com/oss/loki>. [Accessed: 2024].
- [8] Grafana Labs, "Grafana: The open observability platform." [Online]. Available: <https://grafana.com>. [Accessed: 2024].
- [9] Resilience4j Authors, "Resilience4j fault tolerance library." [Online]. Available: <https://resilience4j.readme.io>. [Accessed: 2024].
- [10] A. Basiri et al., "Chaos engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, May/Jun. 2016.
- [11] Codecentric AG, "Spring Boot Admin." [Online]. Available: <https://github.com/codecentric/spring-boot-admin>. [Accessed: 2024].
- [12] Springdoc Authors, "Springdoc OpenAPI." [Online]. Available: <https://springdoc.org>. [Accessed: 2024].

- [13] Docker Inc., "Docker Compose." [Online]. Available: <https://docs.docker.com/compose>. [Accessed: 2024].
- [14] W. J. Conover, *Practical Nonparametric Statistics*, 3rd ed. New York, NY: Wiley, 1999.
- [15] K. R. Beerreddy, "Self-Healing Streaming Architecture: Resilience Patterns for Event-Driven Microservices," *Journal of Computer Science and Technology Studies*, vol. 7, no. 8, pp. 254–268, Jul. 2025.
- [16] S. Purella, "Microservices and Event-Driven Architectures in High-Volume Financial Systems," *Sarcouncil Journal of Engineering and Computer Sciences*, vol. 4, no. 7, pp. 851–860, 2025.
- [17] C. Majors, L. Fong-Jones, and G. Sheffield, *Observability Engineering*. O'Reilly Media, 2022.
- [18] M. S. Islam et al., "Anomaly Detection in Large-Scale Cloud Systems: An Industry Case and Dataset," *arXiv preprint*, Jan. 2025.
- [19] N. H. Minsky, "On Conditions for Self-Healing in Distributed Software Systems," in *Proc. Workshop on Algorithms and Architectures for Self-Managing Systems*, 2003.
- [20] T. Wong, M. Wagner, and C. Treude, "Self-Adaptive Systems: A Systematic Literature Review," *Information and Software Technology*, 2022.
- [21] A. Pecchia et al., "Industry practices and event logging," in *Proc. IEEE/ACM 37th Int. Conf. Software Engineering*, 2015, pp. 169–178.
- [22] M. Cinque, D. Cotroneo, and A. Pecchia, "Event logs for software failure analysis," *IEEE Trans. Software Engineering*, vol. 39, no. 6, pp. 806–821, Jun. 2013.
- [23] J. Humble and D. Farley, *Continuous Delivery*. Addison-Wesley, 2010.
- [24] J. vom Brocke, A. Hevner, and A. Maedche, *Introduction to Design Science Research*, 2021.
- [25] T. Abiona, "Observability and Monitoring in Microservice Architectures for Insurance Firms," 2025.
- [26] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, 2014.
- [27] S. Brazil, *Prometheus: Up & Running*. O'Reilly Media, 2018.

[28] A. Wiggins, "The Twelve-Factor App," 2011. [Online]. Available: <https://12factor.net/>.